Miniproject 2, Intensive
Classification of Handwritten Digits

September 2026

Introduction

Classification of handwritten digits is a standard problem in pattern recognition. A typical application is the automatic reading of zip codes on envelopes. This miniproject aims to practice a classification technique using SVD. The problem is as follows:

- You are given a set of manually classified digits, i.e., a set of hand-written digits, for which you know which digit is shown. This is the *training set*. You use it to develop, i.e. *train*, your algorithm
- Based on the training data, you develop an algorithm, for classifying digits.
- To test how good the algorithm performs, you use the *test set*: a set of digits for which you know which numbers they show (so that you can compare) but your algorithm does not get that information. Based on how well it recognizes the digits, its performance is evaluated.

Background

Representing images as matrices

In Figure 1 a sample of handwritten digits $0, 1, \ldots, 9$ is illustrated. The images are downloaded from [1].



Figure 1: A sample of handwritten digits $0, 1, \ldots, 9$.

[1] `https://www.nist.gov/itl/products-and-services/emnist-dataset`

An image in grayscale can be represented by a $p \times q$ matrix M whose (i, j)-th entry corresponds to the brightness of the pixel (i, j). The storage of this matrix requires pq locations. See Figure 2 for an example.

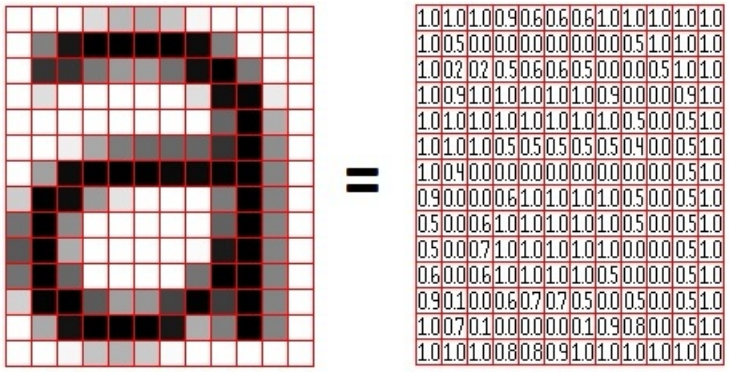


Figure 2: Left image: A grayscale image of the letter 'a', subdivided into blocks of 14 rows and 12 columns. Right image: The intensity of the grayscale is defined by $\xi \in [0, 1]$, where $\xi = 1$ corresponds to pure white, and $\xi = 0$ corresponds to pure black. The block on the ith row and jth column has the corresponding value ξ_{ij}, which relates to the grayness in the corresponding block in the left image.

In your data sets, each image is a so called flattened vector of size $28^2 = 784$ which corresponds to a 28×28 ($p = 28$ and $q = 28$) matrix. For example, for a matrix $C \in \mathbb{R}^{3\times3}$ we would have

$$C = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix} \xrightarrow{\text{Stack columns}} \begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \\ c_{12} \\ c_{22} \\ c_{32} \\ c_{13} \\ c_{23} \\ c_{33} \end{bmatrix}. \tag{0.1}$$

For each kind of digit $d = 0, 1, \ldots, 9$, the training set consists of n handwritten digits. We refer to them as *training images* for digit d. We now define the matrix A_d, where the columns of A_d are the training images of type d. Thus A_d is a $m \times n$ matrix, with $m = 784$:

$$A_d = \begin{bmatrix} \vdots & \vdots & & \vdots \\ \text{Image 1} & \text{Image 2} & \cdots & \text{Image n} \\ \vdots & \vdots & & \vdots \end{bmatrix}.$$

Usually, $n \geqslant m$ although the case $n < m$ is also possible. We refer to the matrix A_d as the *training matrix for digit d*. So, we will have a total of 10 training matrices A_d, each corresponding to one of the digits $d = 0, 1, \ldots, 9$.

SVD-based classification

Assume that A is one of the matrices A_d. The columns of A span a linear subspace in $\mathbb{R}^{784}$. However, most columns of A are nearly linearly dependent as they represent different handwritten styles for the same digit. One can use SVD $A = U\Sigma V^T$ to obtain an orthogonal basis for the column space of A (or $range(A)$) via the columns of matrix U. Then any test image of that kind can be well represented in terms of the orthogonal basis $\{\mathbf{u}_1, \mathbf{u}_2, \ldots, \mathbf{u}_m\}$. We have

$$A = U\Sigma V^T = \sum_{j=1}^{m} \sigma_j \mathbf{u}_j \mathbf{v}_j^T,$$

thus the ℓ-th column of A (the ℓ-th training digit) can be represented as

$$\mathbf{a}_\ell = \sum_{j=1}^{m} (\sigma_j v_{j\ell}) \mathbf{u}_j$$

which means that the coordinates of image $\mathbf{a}_\ell$ in terms of orthogonal basis $\{\mathbf{u}_1, \ldots, \mathbf{u}_m\}$ are $\sigma_j v_{j\ell}$. If we let $x_j = \sigma_j v_{j\ell}$ then

$$\mathbf{a}_\ell = \sum_{j=1}^{m} x_j \mathbf{u}_j = U\mathbf{x}$$

This means that solving the least squares problem

$$\min_{\boldsymbol{x}} \|U\mathbf{x} - \mathbf{a}_\ell\|_2$$

results in an optimal vector $\mathbf{x} = [\sigma_1 v_{1\ell}, \ldots, \sigma_m v_{m\ell}]^T$.

As pointed out before, most columns of A are nearly linearly dependent as they represent different handwritten styles for the same digit. If we translate this to columns of U, this means that a few leading columns of U should capture the subspace well. For this reason the orthogonal vectors $\mathbf{u}_1, \mathbf{u}_2, \ldots,$ are called *singular images* (see Task 2 below). Consequently, we can use a k-rank approximation of A where $k \ll m$. In this case each $\mathbf{a}_\ell$ can be well represented by orthogonal basis $\{\mathbf{u}_1, \ldots, \mathbf{u}_k\}$ in a least squares sense with a small residual.

Now, let δ be a digit of size $m = 784$ outside the training set (which is called a *test digit*). It is reasonable to assume that δ can be well represented in terms of singular images $\{\mathbf{u}_1, \ldots, \mathbf{u}_k\}$ with a relatively small residual. We must compute how well δ can be represented in the 10 different bases, each corresponding to one of the digits $0, 1, \ldots, 9$. This can be done by computing the residual vector in the least squares problems of the type

$$\min_{\boldsymbol{x}} \|U_k\mathbf{x} - \delta\|_2 \tag{0.2}$$

where U_k is a $m \times k$ matrix consisting of the first k columns of U.
One can, however, show that the solution of least squares problem (0.2) is $\mathbf{x} = U_k^T\delta$ and the residual is

$$residual = \|(I - U_k U_k^T)\delta\|_2. \tag{0.3}$$

Hence, to compute the residual, you do not need to solve least squares problems, but just evaluate the formula (0.3).

To sum up, a SVD-based classification algorithm of handwritten digits consists of the following steps:

1. For each digit $d = 0, 1, 2, \ldots, 9$, stack *training* images in front of each other and form a matrix A_d (so 10 big matrices).

2. **Training:** Compute the SVD of each $A_d = U\Sigma V^T$ (i.e. 10 SVD) and then let the columns of U_k be the first k columns of U and use them as an approximation space.

3. **Classification:** For a given test digit δ, compute its residual in all ten bases using equation (0.3) above. Classify δ as the digit d corresponding to the smallest residual.

To Do

To get started, download the *training* set `TrainDigits.npy` consisting of 240000 images (24000 images per any digit), and the *test* set `TestDigits.npy` consisting of 40000 images (4000 images per digit) as a zip-file from the Miniproject page in Studium.
The zip-file also contains *label* files `TrainLabels.npy` and `TestLabels.npy` .These contain digits (0 to 9) and show what digit image is stored in the corresponding column in `TrainDigits` and `TestDigits` respectively. The training labels will be used to learn and the test labels to verify your algorithm.

1. For training matrices of digits 3 and 8, compute the SVD and plot the singular values (one plot for each digit). You only need to use the first 400 images (out of total 24000), so that each training matrix will be of size 784×400. Use linear scales for both axes in your plots. Report your observation and try to justify it. Also plot the first three singular images $\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3$ for digits 3 and 8, separately (six plots), and report your observation. To plot a singular image, you can modify the code given under item 1 under *Programing tips and help* below.

2. Implement the above SVD-based classification algorithm in Python. Use only $n = 400$ training images for each digit (ten matrices each of size 784×400). However, test the algorithm on all 40000 test images and compare your classifications with the exact labels `TestLabels.npy`. Use different values $k = 5, 6, \ldots, 15$ (number of basis functions), and report the percentage of the success of this SVD-based algorithm for each digit with different k values *separately*. If your implementation is correct, you should get some results similar to In Figure 3 below.

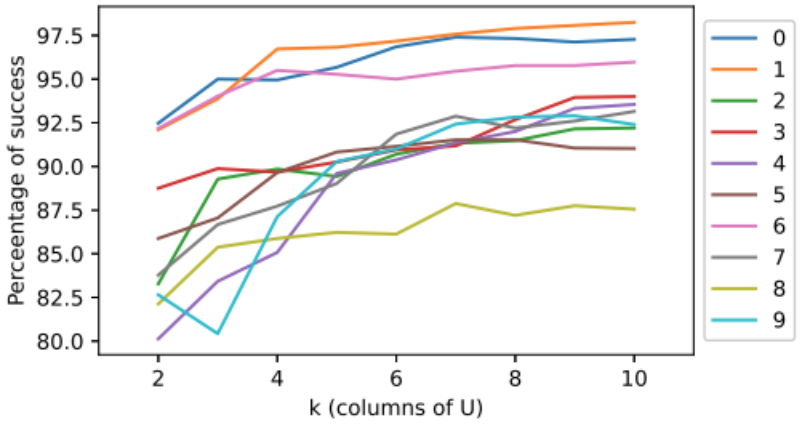


Figure 3: Image taken from the last example in *Davoud Mirzaei: LecturNotes, Numerical Linear Algebra: Least Squares, QR and SVD* (readings in this course)

Programming tips and help

1. To load images you can write, e.g.,

   ```
   TrainMat = np.load('TrainDigits.npy')
   TrainLab = np.load('TrainLabels.npy')
   ```

   Be sure that .npy files are copied in the directory you are working in.
   To plot the digits you need to reshape the columns into 28×28 matrices using `numpy.reshape` and then use `imshow` command to plot the digit. For example here we plot the first digit in the training set:

   ```
   import numpy as np
   import matplotlib.pyplot as plt
   TrainMat = np.load('TrainDigits.npy')
   d = TrainMat[:,0]              # The first digit in the training set
   D = np.reshape(d, (28, 28)).T  # Reshaping a vector to a matrix
   plt.imshow(D, cmap ='gray')    # Plot of the digit
   ```

2. If `a` is a Numpy array and `i` is a scalar value then `a==i` performs an element-wise comparison between them. In other words, if we do `index = (a==i)`, then `index` will be a vector with boolean values `True` or `False`. For example, to extract the first 400 training images for digit 3, we can write

   ```
   index = (TrainLab == 3);  # find train digits of type 3
   A3 = TrainMat[:,index[0]] # all train digits of type 3
   A3 = A3[:,0:400]          # the first 400 train digits of type 3
   ```

   If we then do `np.sum(index)`, we will get the number of Trues in `index`. And, you can use boolean arrays to index other Numpy arrays of suitable size.

3. You must precompute all ten SVDs and store U_k matrices for all ten digits (store U_k for maximum k, here 15, and then pick up smaller subsets for $k = 5, 6, \ldots, 15$. You can use e.g. a dictionary for storing). Then use the precomputed matrices for all 40000 test digits.

4. The equation (0.3) above are formulated for a single test vector δ (one "flattened" test image). But if you use that approach in your code, it will take a very long time to run. In contrast, matrix-matrix multiplications are much more efficient on modern computers. To take full advantage of that, simply replace vector δ in Equation (0.3) with the entire test matrix. Then you can apply `np.linalg.norm(..,axis = 0)` to get residuals for all test points, in one vector of length 40000.

5. If A is a matrix, the function `np.argmin(A, axis=0)` returns a vector containing the row number of the minimal value in each column of A. If you store the residual vectors for all 10 digits in one matrix, you can use this function to efficiently predict digits from residuals.
   In general, take note of this "axis" argument in many Numpy functions: it lets you perform an operation along a specific dimension of a matrix (that is, along rows or columns).